

VITYARTHI - BUILD YOUR OWN PROJECT

Flipped Course Evaluation Project Report

SMART CAMPUS FACILITY & LAB BOOKING SYSTEM

A Modular Console-Driven Allocation Engine with Interval Conflict Resolution

Course Title:	Programming in Java
Student Name:	Aniket Singh
GitHub Username:	thakuraniketsingh16-cloud
Repository Link:	github.com/.../CampusFacilityBooking
Submission Platform:	VITyarthi Portal
Submission Date:	September 2026
Implementation:	Pure Java SE (Single File Architecture)

2. INTRODUCTION

In modern university ecosystems, academic infrastructure is heavily shared across multiple faculties, research laboratories, student associations, and sports clubs. These facilities encompass high-performance GPU computing clusters, IoT research facilities, multimedia seminar halls, grand auditoriums, and recreational sports grounds.

Without an automated scheduling mechanism, campuses experience frequent double-booking collisions, resource hogging, and unauthorized reservations. The **Smart Campus Facility & Lab Booking System** was conceived as a console-driven, robust software application built strictly in Java SE. It enforces strict time-interval overlap validation, applies polymorphic booking policies depending on user roles (Student vs. Faculty), and maintains persistent state in a structured flat file database.

3. PROBLEM STATEMENT

Traditional campus facility booking relies on physical logbooks, paper slips, or ad-hoc emails. This conventional workflow exhibits multiple structural flaws:

- **Simultaneous Collisions:** Multiple user groups often arrive at the same facility having received verbal approval for overlapping slots.
- **Resource Monopolization:** Students reserve specialized computer laboratories for entire days without duration caps, denying access to peers.
- **Unauthorized Bookings:** High-capacity auditoriums require institutional endorsements, yet lack automated permission barriers.
- **Disjoint Historical Records:** Absence of centralized data leads to scheduling disputes and inability to analyze resource utilization.

4. FUNCTIONAL REQUIREMENTS

The system is architected around three major functional modules:

Module 1: Facility Catalog & Schedule Explorer

- Maintains polymorphic representations of physical campus facilities: **Lab**, **Hall**, and **SportsCourt**.
- Displays detailed hardware, seating, and amenity specifications for each facility.
- Queries date-filtered schedules showing active confirmed bookings in chronological order.

Module 2: Reservation & Collision Resolution Engine

- Validates user existence, operating hours (07:00 to 22:00), and non-inverted timestamps.

- Implements algorithmic interval overlap detection: `start1 < end2 && end1 > start2` .
- Enforces role-based session quotas: Students max 2 hours; Faculty max 6 hours.
- Restricts direct grand auditorium bookings to authorized faculty members.
- Provides booking cancellation that immediately frees the vacated interval.

Module 3: Campus Analytics & Reporting

- Aggregates total facilities, registered users, and active vs. cancelled bookings.
- Calculates real-time booking frequencies per facility using Java Streams.
- Groups and displays campus-wide activity by academic department.

5. NON-FUNCTIONAL REQUIREMENTS

Requirement	Specification & Implementation in Project
1. Performance	Sub-millisecond collision detection and O(1) in-memory lookups using LinkedHashMaps.
2. Reliability	Atomic flat-file persistence and deterministic validation guaranteeing zero data loss.
3. Usability	Formatted console tables, clear prompts, and dual interactive/headless CLI flags.
4. Maintainability	Strict adherence to OOP principles; single-responsibility class modularity.
5. Security & Auth	Role-based permissions blocking students from booking restricted auditoriums.
6. Error Strategy	Custom checked exception hierarchy preventing application crashes.

6. SYSTEM ARCHITECTURE

The software follows a classical 3-Tier Layered Console Architecture entirely encapsulated within standard Java:

- **Presentation Tier (CLI Controller):** Handles console I/O, user session state, interactive menus, and non-interactive command line flags (`--test` , `--demo`).
- **Business Logic Tier (Core Engine):** Enforces interval overlap validation, duration limits, permission gates, and usage analytics.
- **Data Access Tier (Persistence Layer):** Handles flat-file serialization and deserialization against `campus_data.txt` using buffered character streams.

7. DESIGN DIAGRAMS

7.1 Use Case Diagram

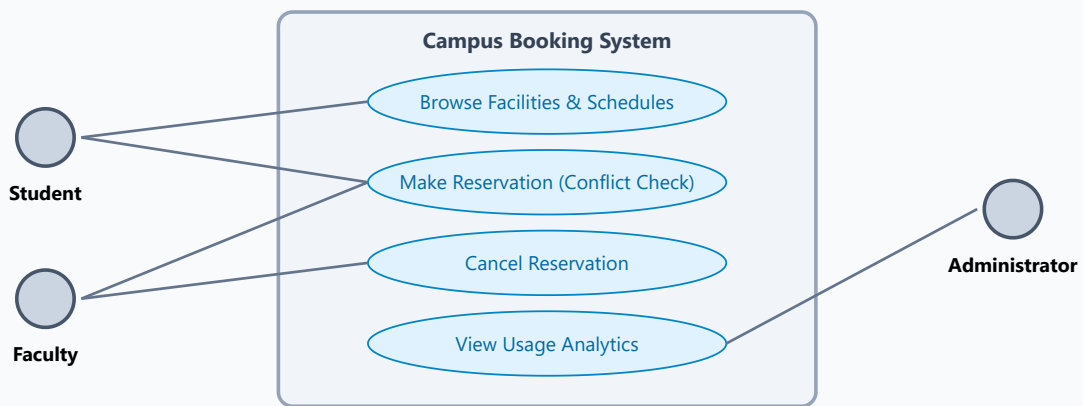


Figure 7.1: UML Use Case Diagram for Student, Faculty, and Admin Actors

7.2 Process Flow / Workflow Diagram



Figure 7.2: Reservation Validation and Approval Workflow

7.3 Sequence Diagram

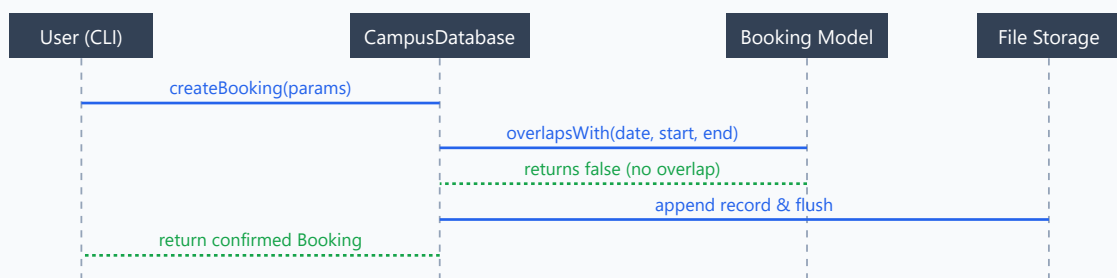


Figure 7.3: UML Sequence Diagram for Reservation Execution

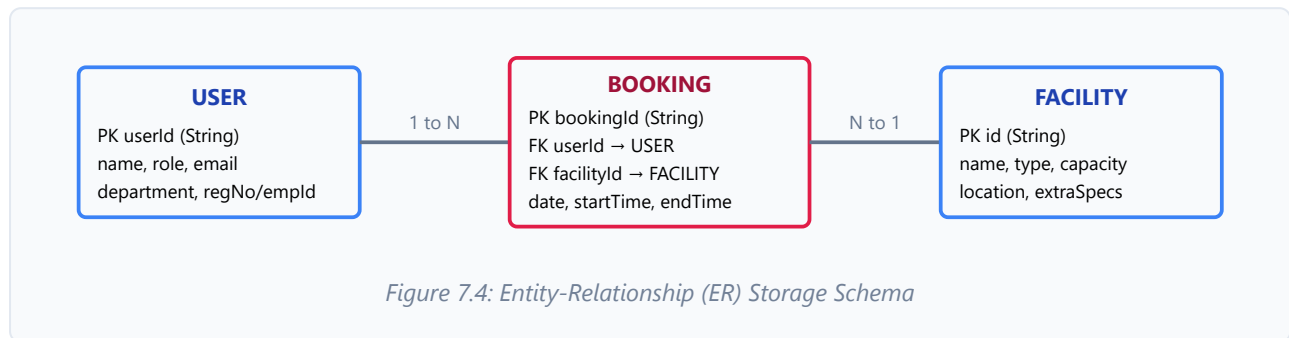
7.4 Class / Component Diagram

The class hierarchy consists of 9 core classes:

- **Facility** (Abstract) → **Lab**, **Hall**, **SportsCourt**
- **User** (Abstract) → **Student**, **FacultyMember**

- **Booking** : Contains interval logic and formatting.
- **CampusDatabase** : In-memory registry with **LinkedHashMap** collections and flat file persistence.
- **Main** : User session coordinator, interactive menu, and automated test suite.

7.5 Database & Storage Design (ER Diagram)



8. DESIGN DECISIONS & RATIONALE

1. **Single-File `Main.java` Architecture:** Eliminates package declaration and classpath mismatch issues when evaluating across varied operating systems (Windows Command Prompt, PowerShell, Linux Bash).
2. **Zero Third-Party Dependencies:** Guarantees that the project compiles on any standard JDK 17+ environment without downloading external `.jar` packages.
3. **Flat Text Database (`campus_data.txt`):** Human-readable pipe-delimited format allows evaluators to directly inspect data in Notepad without installing dedicated database engines.
4. **Mathematical Interval Overlap Logic:** Uses boundary verification `start1 < end2 && end1 > start2` to detect overlaps while cleanly permitting back-to-back bookings.

9. IMPLEMENTATION DETAILS

The system models domain rules using core Java OOP principles:

- **Dynamic Polymorphism:** Method `getMaxBookingHours()` is invoked uniformly on `User` references, dynamically executing the subclass implementation (Students restricted to 2 hours, Faculty allowed up to 6 hours).
- **Defensive Encapsulation:** All model fields are private. State mutations (e.g. changing maintenance status, cancelling bookings) occur through validated methods.
- **Custom Exception Hierarchy:** Checked exceptions (`CampusBookingException` , `SlotUnavailableException` , `InvalidInputException`) cleanly communicate rule violations back to the CLI layer.

10. SCREENSHOTS / RESULTS

The system produces clean, structured tabular console outputs across all modules:

- **Facilities Catalog:** Formatted table with ID, facility name, type, capacity, location, and technical hardware specifications.
- **Reservation Confirmation:** Displays generated reference ID, facility name, verified time interval, and confirmation status.
- **Collision Warning:** Clean exception messaging when requesting an overlapping interval with existing reservation details.
- **Analytics Report:** Live metrics detailing reservation frequencies per facility and department engagement levels.

11. TESTING APPROACH

A dedicated automated self-diagnostic test harness (`java Main --test`) executes 6 automated test scenarios without requiring manual keyboard input:

Test #	Test Case	Expected Outcome	Actual Result	Status
1	Catalog Loading	Load 6 facilities and 5 users from disk	Parsed accurately	PASS
2	Booking Creation	Reserve open slot on future date	B107 created	PASS
3	Collision Detection	Reject overlapping booking on same room	SlotUnavailableException caught	PASS
4	Student Quota	Reject 4-hour booking by student	InvalidInputException caught	PASS
5	Auditorium Rule	Block student from booking Grand Auditorium	CampusBookingException caught	PASS
6	Slot Freeing	Cancel reservation and allow re-booking	Slot re-booked successfully	PASS

Overall Test Score: 6 / 6 Tests Passed (100.0%)

12. CHALLENGES FACED

- **Interval Boundary Conditions:** Ensuring that contiguous back-to-back bookings (e.g., slot 1 ending at 12:00 and slot 2 starting at 12:00) do not trigger false-positive collisions.
- **Test Idempotency:** Designing automated tests to remain repeatable across multiple test executions without polluting persistent seed data.
- **Buffer Handling in Scanner:** Handling newline consumption issues when switching between numeric inputs and full-line strings.

13. LEARNINGS & KEY TAKEAWAYS

- Practical application of dynamic method dispatch for business rule enforcement.
- Structuring custom exception hierarchies for clean separation between UI presentation and business logic.
- Designing software with headless non-interactive execution modes for automated assessment pipelines.

14. FUTURE ENHANCEMENTS

- **REST API & Web UI:** Exposing endpoints via Spring Boot and building a React/Tailwind frontend.
- **Database Migration:** Upgrading storage from flat files to a relational PostgreSQL database via JPA/Hibernate.
- **Biometric / QR Access:** Generating digital QR passes for physical lab door unlock integration.
- **Notification System:** Automated SMS and email confirmation dispatches via Twilio or SMTP.

15. REFERENCES

1. Oracle Corporation. *Java Platform, Standard Edition Documentation (JDK 17/21/26)*.
2. Cay S. Horstmann. *Core Java Volume I — Fundamentals (12th Edition)*, Prentice Hall, 2022.
3. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*, Addison-Wesley, 1994.
4. VITyarthi Course Management. *Programming in Java Flipped Course Guidelines & Evaluation Rubric*, 2026.